

SSN College of Engineering – Kalavakkam

Department of Computer Science and Engineering

ICS1513 – Computer Networks Laboratory

Experiment 5: Simulation of Domain Name Resolution Using UDP Socket Programming

Name: Saravana Senthilkumar | **Roll No:** 3122247001057 | **Date:** 05.08.2026

Objective

To develop a connectionless DNS client-server application in C using UDP sockets that simulates domain name resolution, client-side caching, and robust error handling (NXDOMAIN, timeout, and format validation), while evaluating UDP's suitability for DNS services in terms of overhead, latency, and reliability.

Suitability Analysis of UDP for DNS Services

1. Communication Overhead:

- UDP headers are minimal (8 bytes) compared to TCP headers (20–60 bytes).
- Connection establishment (3-way handshake) and tear-down overhead are completely eliminated, saving bandwidth and network resource allocations for short query-response pairs.

2. Latency:

- Resolution occurs in a single Round Trip Time (1 RTT) under normal conditions (**Client Request** → **Server Response**), whereas TCP requires at least 3 RTTs for handshake, data exchange, and connection teardown.
- This drastic reduction in latency is critical for standard web browsing, where dozens of DNS resolutions are executed per web page load.

3. Reliability & Error Handling:

- UDP is inherently connectionless and unacknowledged; packet loss can result in lost queries or responses.
- To achieve reliability, DNS applications implement application-layer timeout mechanisms (using socket options like `SO_RCVTIMEO`) and retransmissions. If a query times out, the client re-transmits to an alternate or primary server.
- UDP is optimal for standard DNS messages under 512 bytes (or EDNS0 limits). For large payload responses or zone transfers (AXFR), DNS falls back to TCP.

Algorithm

DNS Server Algorithm

1. Create a UDP datagram socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
2. Configure the server address structure (`AF_INET`, `INADDR_ANY`, Port 8080) and bind the socket using `bind()`.
3. Display the initialized local DNS dictionary table (Hostname → IP mapping).
4. Enter an infinite server listening loop:
 - Receive host query string from a client via `recvfrom()`.
 - Strip trailing newlines or whitespace characters.
 - Validate the domain format using `is_valid_domain()` (checks string length, presence of dot, valid alphanumeric/hyphen characters).
 - If format is invalid, send "ERROR:INVALID_FORMAT" back to the client.
 - If format is valid, query the local DNS dictionary table:
 - If matched, format response string as "IP:<ip_address>".
 - If not found, format response string as "ERROR:NXDOMAIN".
 - Transmit response datagram to client using `sendto()`.

DNS Client Algorithm

1. Initialize local client-side cache structure (capacity of 16 entries).
2. For each query domain:
 - **Cache Check:** Search local client cache using `cache_lookup()`. If present, return cached IP immediately (**CACHE HIT**) without network call.
 - **Cache Miss:** If not found, create a UDP socket `socket(AF_INET, SOCK_DGRAM, 0)`.
 - Set socket receive timeout to 3 seconds using `setsockopt(..., SO_RCVTIMEO, ...)`.
 - Send target hostname datagram to server (127.0.0.1:8080) using `sendto()`.
 - Wait for reply using `recvfrom()`.
 - If `recvfrom()` fails with `EAGAIN` or `EWOULDBLOCK`, output a timeout error message.
 - If a response is received:
 - Parse "IP:<ip>": Print resolved IP and store entry in local cache via `cache_store()`.
 - Parse "ERROR:NXDOMAIN": Display non-existent domain error.
 - Parse "ERROR:INVALID_FORMAT": Display invalid format error.
3. Close socket after resolving query.
4. Display final cache state at end of execution.

Server Code (dns_server.c)

/*

```

* dns_server.c
* SSN College of Engineering – UDP DNS Simulation
* Compile: gcc dns_server.c -o server
* Run:    ./server
*/

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <ctype.h>

```

```

#define PORT      8080
#define BUF_SIZE  256
#define TABLE_SIZE 10

```

```

/* — DNS Table (hostname → IP)

```

```

*/
typedef struct {
    char hostname[BUF_SIZE];
    char ip[16];
} dns_entry;

```

```

dns_entry dns_table[TABLE_SIZE] = {
    {"google.com",  "142.250.182.46"},
    {"youtube.com", "142.250.72.238"},
    {"facebook.com", "157.240.241.35"},
    {"instagram.com", "157.240.241.174"},
    {"twitter.com",  "104.244.42.193"},
    {"github.com",   "140.82.121.4"},
    {"ssn.edu.in",   "203.197.132.10"},
    {"amazon.com",   "205.251.242.103"},
    {"wikipedia.org", "208.80.154.224"},
    {"openai.com",   "172.64.155.209"},
};

```

```

/* — Helpers

```

```

— */

```

```

/* Returns 1 if the query string is a valid-looking domain name */

```

```

int is_valid_domain(const char *query) {
    int len = strlen(query);
    if (len == 0 || len > 253) return 0;

    /* Must contain at least one dot */
    int has_dot = 0;
    for (int i = 0; i < len; i++) {
        char c = query[i];
        if (c == '.') { has_dot = 1; continue; }
        if (!isalnum(c) && c != '-' && c != '_') return 0;
    }
    return has_dot;
}

/* Look up a hostname in the DNS table; returns IP string or NULL */
const char *lookup(const char *hostname) {
    for (int i = 0; i < TABLE_SIZE; i++) {
        if (strcasecmp(dns_table[i].hostname, hostname) == 0)
            return dns_table[i].ip;
    }
    return NULL;
}

/* — Main


---


_____ */

int main(void) {
    int sockfd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUF_SIZE];

    /* 1. Create UDP socket */
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }

    /* 2. Bind to port */
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

```

```

if (bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    perror("bind");
    close(sockfd);
    exit(EXIT_FAILURE);
}

```

```

printf("DNS Server listening on port %d...\n\n", PORT);
printf("%-25s %s\n", "HOSTNAME", "IP ADDRESS");
printf("%-25s %s\n", "-----", "-----");
for (int i = 0; i < TABLE_SIZE; i++)
    printf("%-25s %s\n", dns_table[i].hostname, dns_table[i].ip);
printf("\nWaiting for queries...\n\n");

```

/* 3. Query-response loop */

```

while (1) {
    memset(buffer, 0, BUF_SIZE);

    /* Receive query from client */
    int n = recvfrom(sockfd, buffer, BUF_SIZE - 1, 0,
        (struct sockaddr *)&client_addr, &client_len);
    if (n < 0) { perror("recvfrom"); continue; }

```

```

    buffer[n] = '\0';
    /* Strip trailing newline if any */
    buffer[strcspn(buffer, "\r\n")] = '\0';

```

```

    printf("[QUERY]   %-25s from %s\n",
        buffer, inet_ntoa(client_addr.sin_addr));

```

```

    char response[BUF_SIZE];

```

/* 4. Validate format */

```

if (!is_valid_domain(buffer)) {
    snprintf(response, BUF_SIZE, "ERROR:INVALID_FORMAT");
    printf("[RESPONSE] %s\n\n", response);
} else {

```

/* 5. Look up in DNS table */

```

    const char *ip = lookup(buffer);
    if (ip) {
        snprintf(response, BUF_SIZE, "IP:%s", ip);
        printf("[RESPONSE] %s → %s\n\n", buffer, ip);
    } else {

```

```

        snprintf(response, BUF_SIZE, "ERROR:NXDOMAIN");
        printf("[RESPONSE] %s → NXDOMAIN (domain not found)\n\n", buffer);
    }
}

/* 6. Send response back to client */
sendto(sockfd, response, strlen(response), 0,
        (struct sockaddr *)&client_addr, client_len);
}

close(sockfd);
return 0;
}

```

Client Code (dns_client.c)

```

/*
 * dns_client.c
 * SSN College of Engineering – UDP DNS Simulation
 * Compile: gcc dns_client.c -o client
 * Run:    ./client <domain>
 *         ./client google.com
 *         ./client unknown.xyz
 *         ./client "bad domain"
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/time.h>
#include <errno.h>

#define SERVER_IP  "127.0.0.1"
#define PORT      8080
#define BUF_SIZE   256
#define TIMEOUT_SEC 3    /* seconds before timeout */
#define CACHE_SIZE 16

/* — Client-side Cache

```

```

typedef struct {
    char hostname[BUF_SIZE];
    char ip[16];
} cache_entry;

cache_entry cache[CACHE_SIZE];
int cache_count = 0;

/* Look up hostname in local cache; returns IP string or NULL */
const char *cache_lookup(const char *hostname) {
    for (int i = 0; i < cache_count; i++) {
        if (strcasecmp(cache[i].hostname, hostname) == 0)
            return cache[i].ip;
    }
    return NULL;
}

/* Store a resolved hostname→IP pair in the cache */
void cache_store(const char *hostname, const char *ip) {
    if (cache_count < CACHE_SIZE) {
        strncpy(cache[cache_count].hostname, hostname, BUF_SIZE - 1);
        strncpy(cache[cache_count].ip, ip, 15);
        cache_count++;
        printf("[CACHE]  Stored: %-25s → %s\n", hostname, ip);
    }
}

/* Print the entire cache for inspection */
void cache_print(void) {
    if (cache_count == 0) {
        printf("(cache is empty)\n");
        return;
    }
    printf("\n%-25s %s\n", "CACHED HOSTNAME", "IP ADDRESS");
    printf("%-25s %s\n", "-----", "-----");
    for (int i = 0; i < cache_count; i++)
        printf("%-25s %s\n", cache[i].hostname, cache[i].ip);
}

/* — DNS Resolution


---


void resolve(const char *domain) {

```

```

printf("\n=====\\n");
printf("Resolving: %s\\n", domain);
printf("=====\\n");

/* 1. Check cache first */
const char *cached_ip = cache_lookup(domain);
if (cached_ip) {
    printf("[CACHE HIT] %s → %s (served from cache, no network call)\\n",
        domain, cached_ip);
    return;
}
printf("[CACHE MISS] Not in cache — querying DNS server...\\n");

/* 2. Create UDP socket */
int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
if (sockfd < 0) { perror("socket"); return; }

/* 3. Set receive timeout (simulates DNS timeout) */
struct timeval tv = { .tv_sec = TIMEOUT_SEC, .tv_usec = 0 };
if (setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv)) < 0) {
    perror("setsockopt"); close(sockfd); return; }

/* 4. Build server address */
struct sockaddr_in server_addr;
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
if (inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr) <= 0) {
    fprintf(stderr, "Invalid server IP\\n"); close(sockfd); return;
}

/* 5. Send the query */
if (sendto(sockfd, domain, strlen(domain), 0,
    (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    perror("sendto"); close(sockfd); return;
}
printf("[SENT] Query sent for: %s\\n", domain);

/* 6. Wait for response */
char response[BUF_SIZE];
memset(response, 0, BUF_SIZE);
socklen_t server_len = sizeof(server_addr);

```

```

int n = recvfrom(sockfd, response, BUF_SIZE - 1, 0,
                (struct sockaddr *)&server_addr, &server_len);

if (n < 0) {
    if (errno == EAGAIN || errno == EWOULDBLOCK)
        printf("[TIMEOUT] No response from server after %d seconds.\n",
                TIMEOUT_SEC);
    else
        perror("recvfrom");
    close(sockfd);
    return;
}

response[n] = '\0';

/* 7. Parse response */
if (strncmp(response, "IP:", 3) == 0) {
    const char *ip = response + 3;
    printf("[RESOLVED] %s → %s\n", domain, ip);
    cache_store(domain, ip);          /* store in cache */

} else if (strcmp(response, "ERROR:NXDOMAIN") == 0) {
    printf("[ERROR]  NXDOMAIN — '%s' does not exist.\n", domain);

} else if (strcmp(response, "ERROR:INVALID_FORMAT") == 0) {
    printf("[ERROR]  INVALID FORMAT — '%s' is not a valid domain name.\n",
           domain);
} else {
    printf("[UNKNOWN] Unexpected response: %s\n", response);
}

close(sockfd);
}

/* — Main
_____ */
int main(int argc, char *argv[]) {
    printf("=====\n");
    printf(" DNS Client – UDP Socket Simulation\n");
    printf(" Server: %s:%d Timeout: %ds\n",
           SERVER_IP, PORT, TIMEOUT_SEC);
    printf("=====\n");

```

```

if (argc < 2) {
    /* Interactive mode */
    printf("Usage: %s <domain> [domain2] ...\n");
    printf("Running demo queries...\n");

    /* Demo: mix of hits, misses, cache hits, and errors */
    resolve("google.com");    /* should resolve */
    resolve("ssn.edu.in");    /* should resolve */
    resolve("google.com");    /* cache hit — no server call */
    resolve("unknown.xyz");    /* NXDOMAIN */
    resolve("bad domain!");    /* INVALID FORMAT */

} else {
    /* Resolve each domain passed as argument */
    for (int i = 1; i < argc; i++)
        resolve(argv[i]);
}

/* Print final cache state */
printf("\n--- Final Cache Contents ---\n");
cache_print();

return 0;
}

```

Code Output & Execution Screenshots

![DNS Query Execution Output](Pasted image.png)

Terminal Logs:

DNS Server Output Terminal:

DNS Server listening on port 8080...

HOSTNAME	IP ADDRESS
-----	-----
google.com	142.250.182.46
youtube.com	142.250.72.238
facebook.com	157.240.241.35
instagram.com	157.240.241.174
twitter.com	104.244.42.193

github.com	140.82.121.4
ssn.edu.in	203.197.132.10
amazon.com	205.251.242.103
wikipedia.org	208.80.154.224
openai.com	172.64.155.209

Waiting for queries...

[QUERY] google.com from 127.0.0.1
[RESPONSE] google.com → 142.250.182.46

[QUERY] ssn.edu.in from 127.0.0.1
[RESPONSE] ssn.edu.in → 203.197.132.10

DNS Client Terminal Execution:

```
[sarav@vivobook-s14-m5406w Exp5]$ ./client google.com ssn.edu.in google.com
```

```
=====
```

```
DNS Client – UDP Socket Simulation
```

```
Server: 127.0.0.1:8080 Timeout: 3s
```

```
=====
```

```
=====
```

```
Resolving: google.com
```

```
=====
```

```
[CACHE MISS] Not in cache — querying DNS server...
```

```
[SENT] Query sent for: google.com
```

```
[RESOLVED] google.com → 142.250.182.46
```

```
[CACHE] Stored: google.com → 142.250.182.46
```

```
=====
```

```
Resolving: ssn.edu.in
```

```
=====
```

```
[CACHE MISS] Not in cache — querying DNS server...
```

```
[SENT] Query sent for: ssn.edu.in
```

```
[RESOLVED] ssn.edu.in → 203.197.132.10
```

```
[CACHE] Stored: ssn.edu.in → 203.197.132.10
```

```
=====
```

```
Resolving: google.com
```

```
=====
```

```
[CACHE HIT] google.com → 142.250.182.46 (served from cache, no network call)
```

--- Final Cache Contents ---

CACHED HOSTNAME	IP ADDRESS
google.com	142.250.182.46
ssn.edu.in	203.197.132.10

Learning Outcomes

1. Implemented a full client-server DNS name resolution service over UDP sockets in C.
2. Understood client-side caching strategies to eliminate redundant network round trips.
3. Handled network exceptions using socket timeouts (SO_RCVTIMEO) and domain name format validation.
4. Analyzed why UDP is preferred for DNS over TCP due to reduced overhead and lower latency.